Divide and conquer

Part I Foundations

Scope and prerequisites

Complete case analysis and recursive contracts establish correctness. Maximum subarrays and Strassen's seven products provide the algorithms. Fixed-constant induction, level sums, and a full master-theorem proof establish costs, with explicit rounding, logarithmic boundaries, and unequal-split assumptions. Prerequisites are recursion, array intervals, matrix-product indexing, and induction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

- Compute maximum contiguous subarrays by divide and conquer.
- Explain Strassen's block construction and subcubic cost.
- Analyze recurrences by substitution, recursion trees, and the master theorem.
- Identify the external Akra–Bazzi assumptions for unequal subproblem sizes and evaluate its integral.

Sections 4.1–4.6 cover maximum subarrays, Strassen, substitution, recursion trees, the master method, and its proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Separate recursive correctness from cost

Divide and conquer splits an input into smaller instances of the same problem, solves them recursively, and combines their answers. Correctness requires that child answers cover the original problem, followed by induction. Costs include division, combination, allocation, and copying at every call.

Use 1-based arrays and inclusive intervals. Time uses unit-cost exact real or integer arithmetic. If integer bit lengths grow, multiplication and addition cannot unconditionally remain O(1). Exact matrix algebra does not itself guarantee floating-point error bounds.

An equation $T(n) = \sum_i T(n_i) + f(n)$ describes complete cost. An inequality with $\leq$ normally supplies only an upper bound. Specify bases, strict shrinking $n_i < n$, and actual split costs.

4.1 The maximum-subarray problem

For $n \geq 1$, return a nonempty contiguous interval [i,j] maximizing $\sum_{k=i}^{j} A[k]$, together with (i,j,sum). Ties may return any maximizing interval unless another rule is specified.

For prices $P_0,\ldots,P_n$ with buying before selling, define $A_i = P_i - P_{i-1}$. Then $P_j - P_{i-1} = \sum_{k=i}^{j} A_k$, so i..j means buying on day i−1 and selling on day j. Allowing no trade additionally permits the empty interval with profit 0. These specifications differ on all-negative inputs.

Enumerating starts and extending endpoints with accumulated sums costs $\Theta(n^2)$ time and O(1) extra space. Recomputing each interval sum from scratch instead costs $\Theta(n^3)$. Prefix sums make interval queries O(1), but leave $\Theta(n^2)$ candidates.

Crossing means suffix plus prefix

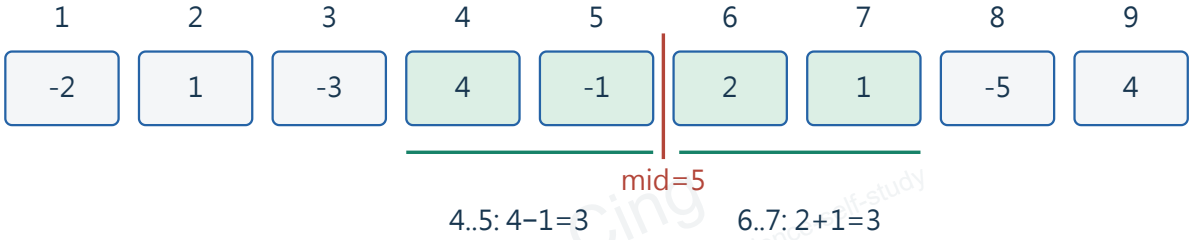
For low<high, let mid=floor((low+high)/2). Every nonempty interval belongs to exactly one class: wholly in low..mid, wholly in mid+1..high, or crossing with i≤mid<j. A crossing interval uniquely consists of left suffix i..mid and right prefix mid+1..j.

The endpoints can be chosen independently, so the best crossing sum is the best nonempty left suffix plus the best nonempty right prefix. Scan each side instead of enumerating endpoint pairs. Combining the halves' maximum subarrays is invalid because they need not reach the midpoint.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Crossing means suffix plus prefix (continued)

Indices and nonempty crossing interval: this call returns A[4..7]



Crossing sum 3+3=6. The left suffix and right prefix must meet at the midpoint.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete crossing scan

```
CROSS(A, low, mid, high):
    left = -infinity; total = 0
    for i = mid downto low:
        total += A[i]
        if total > left: left = total; best_i = i
    right = -infinity; total = 0
    for j = mid+1 to high:
        total += A[j]
        if total > right: right = total; best_j = j
    return (best_i, best_j, left+right)
```

After a left-loop update, total equals the current i..mid sum, and left is the largest suffix sum examined. Initially there is no candidate; negative infinity prevents selecting an empty interval. Each iteration adds exactly one candidate and updates the maximum. At exit every left suffix has been considered. The right proof is identical. Total iterations are (mid−low+1)+(high−mid)=high−low+1.

Recursive maximum subarray and proof

```
MAX-SUBARRAY(A, low, high):
    if low == high: return (low, high, A[low])
    mid = floor((low+high)/2)
    L = MAX-SUBARRAY(A, low, mid)
    R = MAX-SUBARRAY(A, mid+1, high)
    C = CROSS(A, low, mid, high)
    return a tuple with largest sum among L, R, C
```

Use strong induction on interval length. A singleton has only itself. For longer intervals, both nonempty children are smaller, so induction gives the first two class maxima. The crossing proof gives the third. These classes are disjoint and exhaustive; their maximum is globally optimal. Shrinking lengths also prove termination.

The recurrence is $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n) = \Theta(n \lg n)$. Sequential calls use O(log n) stack with constant indices and sums per frame. Copying subarrays requires separate allocation and live-space accounting.

Nine-element crossing trace

Let A=[−2,1,−3,4,−1,2,1,−5,4], low=1, high=9, mid=5.

Left i	Sum A[i..5]	Best sum and start
5	−1	−1, i=5
4	3	3, i=4
3	0	3, i=4
2	1	3, i=4
1	−1	3, i=4

Right prefixes ending at 6,7,8,9 have sums 2,3,−2,2, so the best is 6..7 with sum 3. Crossing 4..7 sums to 6. Left and right optima are 4..4 and 9..9, each with sum 4. Return (4,7,6). For [−8,−2,−5], return (2,2,−2), not zero.

A linear algorithm's state and invariant

Let end_i be the best nonempty interval sum required to end at i. Such an interval is A_i alone or a best interval ending at i−1 extended by A_i. Thus $end_i = \max(A_i, end_{i-1} + A_i)$ and $best_i = \max(best_{i-1}, end_i)$, with $end_1 = best_1 = A_1$. Choosing A_i resets the start to i; extension keeps the earlier start. Whenever best improves, save its start and endpoint i. Induction proves both state definitions, giving Θ(n) time and O(1) extra space including interval recovery.

For the nine-element input, end=[−2,1,−2,4,3,5,6,1,5] and best=[−2,1,1,4,4,5,6,6,6], recovering 4..7. Divide and conquer is not fastest here; it illustrates correctness through complete classes and a combination contract.

4.2 Strassen’ s algorithm for matrix multiplication

For n×n matrices, $C_{ij} = \sum_{k=1}^{n} A_{ik}B_{kj}$. Three loops initialize each output to zero and add products by k. The inner invariant is the sum of the first k products. The n^2 outputs each require n multiply-adds, giving $\Theta(n^3)$ for this procedure, not a lower bound for every matrix-multiplication algorithm.

Write A=[[a,b],[c,d]], B=[[e,f],[g,h]], where letters denote n/2-square blocks. Then C_{11}=ae+bg, C_{12}=af+bh, C_{21}=ce+dg, C_{22}=cf+dh. Preserve factor order; matrix blocks generally do not commute.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Full cost of eight recursive products

Pad equal-order square inputs to a power of two, then retain the original result size. Each call reads its own order from the input view's row count rather than keeping the original n.

```
BLOCK-MULTIPLY(A,B):
    n = A.rows
    if n == 1: return [[A[1,1]*B[1,1]]]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    C11 = BLOCK-MULTIPLY(a,e)+BLOCK-MULTIPLY(b,g)
    C12 = BLOCK-MULTIPLY(a,f)+BLOCK-MULTIPLY(b,h)
    C21 = BLOCK-MULTIPLY(c,e)+BLOCK-MULTIPLY(d,g)
    C22 = BLOCK-MULTIPLY(c,f)+BLOCK-MULTIPLY(d,h)
    return the four blocks assembled into C
```

A view stores an origin, stride, and order, making division O(1). Actual block copies cost $\Theta(n^2)$, still matching four block additions. The recurrence $8T(n/2)+\Theta(n^2)$, with constant base cost, gives $\Theta(n^3)$.

A constant number of additions combines into $\Theta(n^2)$, but eight recursive calls cannot become one T(n/2). The branching factor multiplies at every level and determines n^3 leaves. Absorbing local constants differs from changing tree branching.

All seven Strassen products

Compute seven products recursively; preparation uses elementwise block addition and subtraction.

$$\begin{aligned} M_1 &= (a+d)(e+h), & M_2 &= (c+d)e, \\ M_3 &= a(f-h), & M_4 &= d(g-e), \\ M_5 &= (a+b)h, & M_6 &= (c-a)(e+f), \\ M_7 &= (b-d)(g+h). \end{aligned}$$

Recover $C_{11} = M_1 + M_4 - M_5 + M_7$, $C_{12} = M_3 + M_5$, $C_{21} = M_2 + M_4$, and $C_{22} = M_1 - M_2 + M_3 + M_6$. At n=1 multiply scalars directly. Otherwise divide, compute these products, and assemble the outputs. This M numbering is defined here and must not be mixed with another source's P numbering.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Correctness of all four output blocks

Distributivity and additive cancellation suffice; no multiplication factors change order.

$$\begin{aligned} C_{11} &= (ae + ah + de + dh) + (dg - de) \\ &\quad - (ah + bh) + (bg + bh - dg - dh) = ae + bg, \\ C_{12} &= (af - ah) + (ah + bh) = af + bh, \\ C_{21} &= (ce + de) + (dg - de) = ce + dg, \\ C_{22} &= (ae + ah + de + dh) - (ce + de) \\ &\quad + (af - ah) + (ce + cf - ae - af) = cf + dh. \end{aligned}$$

The scalar base is correct. If the seven smaller products are correct by induction, these identities recover all four blocks of AB. The construction needs addition, subtraction, and distributivity; it does not directly apply to a general semiring without subtraction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Numerical products and reconstruction

Let A=[[2,−1],[3,4]], B=[[5,2],[−2,1]]. Thus a=2,b=−1,c=3,d=4,e=5,f=2,g=−2,h=1.

Product	Substitution	Value
M1	(2+4)(5+1)	36
M2	(3+4)5	35
M3	2(2−1)	2
M4	4(−2−5)	−28
M5	(2−1)1	1
M6	(3−2)(5+2)	7
M7	(−1−4)(−2+1)	5

Outputs are 36−28−1+5=12, 2+1=3, 35−28=7, and 36−35+2+7=10. Direct multiplication gives [[10+2,4−1],[15−8,6+4]]=[[12,3],[7,10]].

Numerical products and reconstruction (continued)

Seven products on the same input: preserve all four output positions

A

2	-1
3	4

B

5	2
-2	1

$M_1=36$; $M_2=35$

$M_3=2$; $M_4=-28$

$M_5=1$; $M_6=7$

$M_7=5$

C

12	3
7	10

$C_{11}=36-28-1+5=12$; $C_{12}=2+1=3$

$C_{21}=35-28=7$; $C_{22}=36-35+2+7=10$

Time, padding, and memory

View splitting is O(1). Preparing product inputs uses 10 block additions/subtractions; reconstructing outputs uses 8. Thus $T(n) = 7T(n/2) + \Theta(n^2)$, T(1)=Θ(1), giving $\Theta(n^{\lg 7})$, lg 7≈2.80735. Larger addition constants can make small matrices slower. A fixed threshold switching to ordinary multiplication preserves the exponent.

Pad arbitrary positive n to $m = 2^{\lceil \lg n \rceil}$, n≤m<2n. The upper-left product remains AB; restrict back to n×n. Since $m^{\lg 7} < 7n^{\lg 7}$, the order is unchanged. Return an empty matrix directly at n=0.

With depth-first execution, each call holds a constant number of current-level blocks, runs seven children sequentially, and releases temporaries promptly. Peak extra space satisfies $S(n) \leq S(n/2) + O(n^2) = O(n^2)$, plus O(log n) stack. Simultaneously retaining all child workspaces gives a different recurrence.

4.3 The substitution method for solving recurrences

Guess a bound, then use strong induction with the same fixed constants at every size. Increasing a constant at each level is invalid.

For $T(n) = 2T(\lfloor n/2 \rfloor) + n, T(1) = 1$, try T(n)≤cn lg n. For n≥4 put m=floor(n/2)≥2. Substitution gives

$$2cm \lg m + n \leq cn \lg(n/2) + n = cn \lg n - (c-1)n \leq cn \lg n$$

for c≥1. But n=1 gives zero on the right and cannot be a base. Instead use n=2,3: T_2=4,T_3=5 are covered by c≥2. At n≥4 the child is at least 2, closing induction. Alternatively cn(lg n+1) covers n=1. Every reached small input needs an induction or explicit base case.

Lower bounds and stronger hypotheses

For power-of-two n in T(n)=2T(n/2)+n,T_1=1, the hypothesis T(n)≥dn lg n yields dn lg n+(1−d)n. Choose 0<d≤1 and cover bases to prove Ω(n log n). Rounded trees below extend beyond powers of two.

For $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1, T_1 = 1$, the guess cn gives cn+1 and fails, although linear growth can still be correct. Strengthen to cn−d. Substitution yields cn−2d+1≤cn−d when d≥1; c≥d+1 covers the base. Indeed c=2,d=1 gives exact 2n−1.

Claiming “cn+n is still O(n)” silently replaces c by c+1 instead of proving the same hypothesis. This would incorrectly prove an n log n recurrence linear.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Variable changes and strict shrinking

For $T(n) = 2T(\sqrt{n}) + \lg n$, start on $n = 2^{2^k}$ with T_2=Θ(1). Let m=lg n,S(m)=T(2^m). Then S(m)=2S(m/2)+m, giving Θ(m log m), hence T(n)=Θ(log n log log n). New m differs from original n.

In T(n)=2T(n/2+17)+n, children shrink only for n>34. Put n=34+x,U(x)=T(34+x), obtaining U(x)=2U(x/2)+x+34. A sufficiently large constant base interval gives Θ(x log x)=Θ(n log n). Integer versions retain rounding errors. Giving only T_1 would let n=34 call itself.

The domains are explicit. Extending these proofs to all n requires rounding analysis, monotone comparison, or suitable bases.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.4 The recursion-tree method for solving recurrences

A node stores only its nonrecursive f(size); children account for recursive work. Storing full T(size) at each node and adding descendants double-counts. Count depth, nodes per level, and leaves separately.

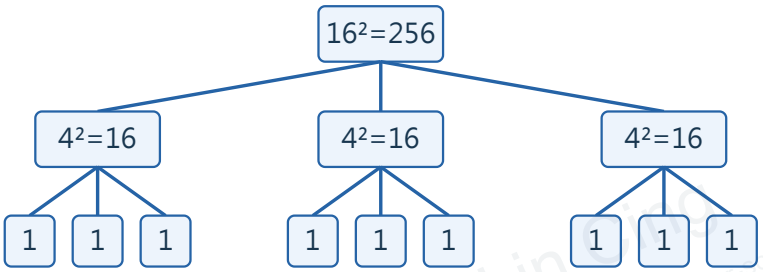
For T(n)=3T(n/4)+n^2,T_1=1,n=4^L, depth j has 3^j nodes of cost $(n/4^j)^2$, hence $n^2(3/16)^j$. Internal depths are 0..L−1; leaves number 3^L=n^($\log_4 3$).

$$T(n) = n^2 \sum_{j=0}^{L-1} (3/16)^j + n^{\log_4 3} = \Theta(n^2).$$

The geometric sum is at most 16/13 and leaf cost at most n^2, giving the upper bound; the root gives lower bound n^2. At n=16, level costs 256,48,9 total 313. The nine leaves already carry base cost, so do not add f(1) again.

4.4 The recursion-tree method for solving recurrences (continued)

Three n/4 children: level-cost ratio 3/16, not 3/4



Depth 0: 1×256=256

Depth 1: 3×16=48

Depth 2: 9×1=9

T(16)=256+48+9=313. Bottom nodes are base cases; charge them once.

Unequal splits: depth and mass per level

For sufficiently large integer n put a=floor(n/3),b=n−a, and T(n)=T(a)+T(b)+cn. Handle fixed small inputs directly. Child sizes sum to n, so surviving mass at each level is at most n, giving cost at most cn.

The longest path shrinks to at most 2n/3+1 each step and has O(log n) depth; there are O(n) leaves. This gives O(n log n). A lower bound cannot assume mass n through the deepest level, because shorter paths stop earlier. For a sufficiently large fixed K absorbing rounding, no path reaches a base in the first floor($\log_3(n/K)$) levels. These complete levels each cost cn, proving Ω(n log n).

If the nonrecursive term is only O(n), the upper bound survives but the lower bound need not. Zero merge cost can leave only Θ(n) leaf work.

4.5 The master method for solving recurrences

Let fixed a≥1,b>1, positive bounded constant-size bases, and nonnegative eventually positive f define T(n)=aT(n/b)+f(n). Put p=log_b a. Start with exact powers; the next section gives rounding conditions.

Case	Required conditions	Result
1	fixed ε>0; f=O(n^(p−ε))	$\Theta(n^p)$
2	$f=\Theta(n^p)$	$\Theta(n^p \log n)$
3	fixed ε>0; f=Ω(n^(p+ε)); af(n/b)≤cf(n) eventually for fixed c<1	Θ(f(n))

Merely smaller or larger than n^p is insufficient; ε must not depend on n. Case 3 regularity makes lower levels payable by root work. Real a can be a recurrence weight; an actual count of a calls is integral.

Complete case classification examples

(a,b,f)	Exponent and check	Result
(9,3,n)	p=2; ε=1; case 1	Θ(n^2)
(1,3/2,1)	p=0; case 2	Θ(log n)
(3,4,n log n)	p≈.7925; ε=.1; 3f(n/4)/f(n)≤3/4	Θ(n log n)
(8,2,n^2)	p=3; ε=1	Θ(n^3)
(7,2,n^2)	p=lg 7; 0<ε≤lg 7−2	Θ(n^(lg 7))

The recurrence 2T(n/2)+n log n fits none of the classical cases. Its log factor grows slower than every fixed n^ε, so polynomial separation fails. Level sums still solve it; an inapplicable theorem does not mean an unsolvable recurrence.

Sum logarithmic boundary cases

For exact powers n=b^L and fixed constant-size bases, let $f(n) = \Theta(n^p(\lg n)^k)$ for fixed real k. Renumber from the leaf end by r=L−j, giving level cost Θ($n^p r^k$).

$$T(n) = \begin{cases} \Theta(n^p(\lg n)^{k+1}) & k > -1, \\ \Theta(n^p \lg\lg n) & k = -1, \\ \Theta(n^p) & k < -1. \end{cases}$$

For k>−1, integral comparison gives $\sum_{r=1}^{L} r^k = \Theta(L^{k+1})$. At k=−1 the harmonic sum is Θ(log L). For k<−1 it converges to a positive constant; leaves still give Θ(n^p). The standard k≥0 extension is a subset. Use logs above the base interval to avoid negative powers of log 1.

Thus 2T(n/2)+n/log n gives Θ(n log log n), not Θ(n); 9T(n/3)+$n^2\log^2 n$ gives Θ($n^2\log^3 n$).

A counterexample without regularity

For $n=2^L, T_1=1$, let $f(n)=n^2$ for even L and n^3 for odd L. With $a=b=2, p=1$, $f \geq n^2$ gives polynomial separation, but T need not be $\Theta(f)$.

For even $L, n \geq 4$, the child's exponent is odd. Then $T(n) \geq 2f(n/2) = n^3/4$, while $f(n)=n^2$, giving unbounded $T/f \geq n/4$. The regularity ratio itself is $2f(n/2)/f(n) = n/4$.

This shows failure after omitting a hypothesis, not failure of the theorem. A one-unit rounding error also need not preserve arbitrary oscillatory f values; conditions must cover actual rounded arguments.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.6 Proof of the master theorem

For $n=b^L, T_1=t>0$, expand L times or induct on tree depth.

$$T(n) = a^L T(1) + \sum_{j=0}^{L-1} a^j f(n/b^j) = tn^p + \sum_{j=0}^{L-1} a^j f(n/b^j).$$

The positive leaf term tn^p gives a lower bound in every case. In case 1, $f(x) \leq Cx^{(p-\varepsilon)}$ bounds depth-j work by $Cn^{(p-\varepsilon)}(b^\varepsilon)^j$. Its ratio $b^\varepsilon > 1$, so the sum is $O(n^{(p-\varepsilon)}(b^\varepsilon)^L) = O(n^p)$. The fixed final levels where the bound may fail also cost $O(n^p)$, absorbing finitely many f values. Adding leaves gives $\Theta(n^p)$.

The geometric sum increases toward leaves. For $a=b=2, \varepsilon=1/2$, the ratio is $\sqrt{2} > 1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Equal-level and root-dominated proofs

Case 2 uses a=bᵖ, making each large-size level cost $a^j \Theta((n/b^j)^p) = \Theta(n^p)$. There are L−O(1)=Θ(log n) such levels. Their two-sided sum gives Θ(nᵖ log n), covering leaf work.

In case 3, repeated regularity gives $a^j f(n/b^j) \le c^j f(n)$ while arguments remain sufficiently large. Their sum is at most f(n)/(1−c). Remaining fixed-size levels and leaves cost O(nᵖ); polynomial separation makes this O(f(n)). The root gives lower bound f(n), proving Θ(f(n)).

Small inputs do not cost O(1) across the whole tree: Θ(nᵖ) such nodes may exist. Charging them to leaf order explicitly prevents omission. Every case gives both bounds.

Rounding errors remain bounded

Along a path, q_0=n and q_{j+1} is floor(q_j/b) or ceil(q_j/b), so $|q_{j+1} - q_j/b| \le 1$. With C≥b/(b−1), induction gives

$$|q_j - n/b^j| \le C,$$

because the next error is at most C/b+1≤C. Choose base interval q≤n_0 with n_0>C+2 and large enough to ensure strict shrinking. Every path reaches a base at depth log_b n+O(1). More than a constant number of levels above leaves, $n/b^j \ge 2C$, so q_j and n/b^j differ by fixed factors.

With a actual children per node, path depths differ by a constant, giving Θ(a^(log_b n))=Θ(nᵖ) leaves. Fixed-size bottom levels cost O(nᵖ). For a=1 this is one path with a constant number of bottom nodes, consistent with the formula.

Conditions for all integer sizes

Case 1 uses $q_j=\Theta(n/b^j)$ above fixed final levels. Powers remain within constant factors even when $p-\varepsilon<0$. Applying f's polynomial upper bound gives the same increasing geometric sum. Add $O(n^p)$ bottom work.

Case 2 similarly has $f(q_j)=\Theta((n/b^j)^p)$. There are $\Theta(\log n)$ complete large-size levels of cost $\Theta(n^p)$; fixed incomplete bottom levels do not change the bound. For logarithmic extensions, $\log q_j$ is comparable to distance r from the bottom, yielding the same three sums.

Case 3 needs regularity on actual children. A sufficient rounded condition is

$$a\max\{f(\lfloor x/b\rfloor),f(\lceil x/b\rceil)\}\le cf(x),\quad c<1.$$

Then total work contracts by c at each level. Add $O(n^p)$ bottom work and use polynomial separation. Bounded argument errors alone do not imply this condition for arbitrary f.

Regularity can imply a polynomial gap

On exact powers, suppose $af(x/b)\le cf(x)$,$0<c<1$, with fixed bottom $f(x_0)>0$. Reversing the inequality k times gives $f(b^kx_0)\ge(a/c)^kf(x_0)$. Since $\log_b(a/c)=p+\log_b(1/c)$, this gives $f(n)=\Omega(n^{p+\varepsilon})$, $\varepsilon=\log_b(1/c)>0$.

This explains formulations that omit a separate gap under positive well-behaved bases. Here the full standard hypotheses remain explicit. The derivation is on exact powers; other domains need control of the terminal interval and its positive lower bound.

The master theorem and integer extension have now been proved. The unequal-ratio tool below is a cited external theorem.

Akra–Bazzi for unequal subproblems

The master theorem uses a equal-size n/b subproblems. Akra–Bazzi permits finitely many fixed ratios. This extends beyond CLRS third-edition Chapter 4. Use the continuous form from MIT 6.042J, with positive bounded bases as sufficient assumptions.

For $0 \le x \le x_0$ require fixed $0 < d_1 \le T(x) \le d_2$. For $x > x_0$ define

$$T(x) = \sum_{i=1}^{k} a_i T(b_i x + h_i(x)) + g(x).$$

Require all of the following.

- Fixed finite k, fixed $a_i > 0$, fixed $0 < b_i < 1$.
- Nonnegative differentiable g, with $|g'(x)| \le D x^C$ eventually for fixed C,D. Examples also have g continuous on the base interval.
- $|h_i(x)| = O(x / \log^2 x)$ for every i.
- A sufficiently large x_0 puts every child in $[0, \rho x]$ for fixed $\rho < 1$, ensuring defined recursion and eventual base entry.

Akra–Bazzi for unequal subproblems (continued)

For the unique real p satisfying $\sum_i a_i b_i^p = 1$, the theorem gives

$$T(x) = \Theta\left(x^p \left[1 + \int_1^x \frac{g(u)}{u^{p+1}}\, du\right]\right).$$

Positive bases pay for leaves. Allowing $T \equiv 0, g \equiv 0$ would invalidate the positive Θ lower bound. This theorem is cited, not proved here. The following arguments derive its root, integral, and applicability.

Rounding versus changed ratios

Floor and ceiling differ from $b_i x$ by at most 1, satisfying the perturbation bound. Fixed additive constants do too. The base interval must still ensure shrinking.

A perturbation εx for fixed $\varepsilon > 0$ is not $O(x/\log^2 x)$. It changes the ratio and requires a new root. The exponent depends on actual sizes, not merely two recursive calls.

Sources using $x/c_i, c_i > 1$ correspond to $b_i = 1/c_i$. Their equations $\sum_i a_i c_i^{-p} = 1$ and $\sum_i a_i b_i^p = 1$ are equivalent. Keep parameter definitions consistent.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Unique exponent and integral

Let $F(p) = \sum_i a_i b_i^p$. Each positive term decreases strictly in p. F tends to infinity at negative infinity and zero at positive infinity. Continuity gives exactly one real root of F=1. $F(0) = \Sigma a_i$ determines whether the root is positive, zero, or negative. It can be fractional; the theorem gives an order, not an exact closed form.

Set $T(x) = 1$ throughout $0 \le x \le 1$ and $T(x) = T(x/3) + T(2x/3) + x$ above 1. Then $a_1 = a_2 = 1, b_1 = 1/3, b_2 = 2/3, g = x$. Substitution gives the unique root $p = 1$ of $(1/3)^p + (2/3)^p = 1$. Thus

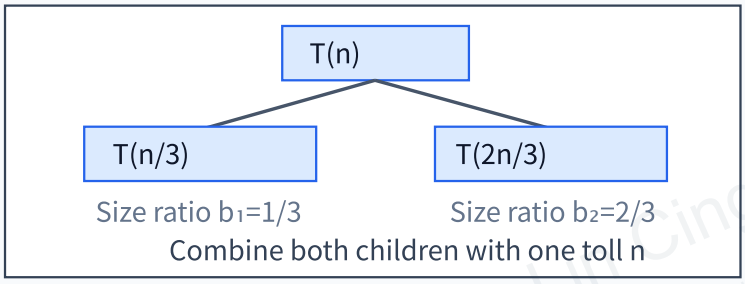
$$T(n) = \Theta\left(n\left[1 + \int_1^n u/u^2\,du\right]\right) = \Theta(n(1 + \ln n)) = \Theta(n \log n).$$

The general theorem supplies both bounds without finite-instance enumeration.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Unique exponent and integral (continued)

Unequal children: T(n)=T(n/3)+T(2n/3)+n



Characteristic equation

$(1/3)^p+(2/3)^p=1$

Strictly decreasing; unique root p=1

$\int_1^n u/u^2\,du = \ln n$

$T(n)=\Theta(n \log n)$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Oversized perturbations change the exponent

For T(n)=T(n/2+n/4)+T(n/2)+n, treating both ratios as 1/2 would predict p=1 and Θ(n log n). But n/4 is not an O(n/log^(1+ε)n) perturbation.

Rewrite as T(n)=T(3n/4)+T(n/2)+n. The equation $(3/4)^p + (1/2)^p = 1$ has numerical root p≈1.507126591639. The integral yields Θ(n^p), not Θ(n log n). Check actual children and perturbation bounds before choosing ratios.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Choosing a recurrence tool

1. For a children of common size n/b, check master-theorem cases and regularity.
2. For finitely many fixed ratios such as n/3 and 2n/3, check Akra–Bazzi, solve p, then integrate.
3. Floor, ceiling, and $O(n/\log^2 n)$ perturbations can retain ratios. Fixed-proportion perturbations require true new ratios.
4. Negative g, missing smoothness, or undefined bases leave theorem hypotheses unmet. Use expansion or direct induction.

The unequal children in T(n/3)+T(2n/3)+n are precisely what the ordinary master theorem cannot directly express.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 4.

M. Akra and L. Bazzi. "On the Solution of Linear Recurrence Equations." *Computational Optimization and Applications* 10(2), 1998. doi:10.1023/A:1018353700639.

K. Mehlhorn. *A Master Theorem for Recurrences*. Max Planck Institute for Informatics, 2008, Theorems 1–3.

MIT 6.042J, *Mathematics for Computer Science*, Fall 2010, § 10.4.3, Theorem 10.4.1. Official notes. This chapter uses positive bounded bases as sufficient assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study